



ASSIGNMENT NO.2

OBJECT ORIENTED PROGRAMMING

SUBMITTED BY:

Vigyat Manktala

1024150373

2034

1. Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members(properties):

Name

Roll No

Degree

Hostel

CurrentCGPA

Member Function(behavior):

addDetails(); updateDetails();

updateCGPA(); updateHostel();

displaydetails();

CODE:

```
#include <iostream>
```

```
#include <string> using
```

```
namespace std;
```

```
struct Student
```

```
{
```

```
    string name;
```

```

    int roll_no;   float cgpa;   string
degree;   string hostel;

    void addDetails()
    {
        cout << "Enter name: ";       cin
>> name;

        cout << "Enter roll number: ";
cin >> roll_no;

        cout << "Enter CGPA: ";       cin
>> cgpa;

        cout << "Enter degree: ";       cin
>> degree;

        cout << "Enter hostel: ";
cin >> hostel;
    }

    void updateDetails()
    {
        cout << "Enter new name: ";
cin >> name;

        cout << "Enter new roll number: ";
cin >> roll_no;

```

Output

```

Enter name: Saanvi
Enter roll number: 13
Enter CGPA: 10
Enter degree: btech
Enter hostel: Q

Student Details:
Name: Saanvi
Roll No: 13
CGPA: 10
Degree: btech
Hostel: Q
Enter new name: ROY
Enter new roll number: 14
Enter new CGPA: 10
Enter new degree: llb
Enter new hostel: A

=== Code Execution Successful ===

```

```
        cout << "Enter new CGPA: ";  
cin >> cgpa;
```

```
        cout << "Enter new degree: ";  
cin >> degree;
```

```
        cout << "Enter new hostel: ";  
cin >> hostel;  
    }
```

```
void displayDetails()  
{  
    cout << "\nStudent Details:\n";  
    cout << "Name: " << name << endl;  
    cout << "Roll No: " << roll_no << endl;  
    cout << "CGPA: " << cgpa << endl;    cout  
<< "Degree: " << degree << endl;    cout  
<< "Hostel: " << hostel << endl;  
}  
};
```

```
int main()  
{  
  
    s.addDetails();  
    s.displayDetails();
```

```
s.updateDetails(); return  
0;  
}
```

2. Differentiate between private and public access/scope. Perform the question no. 1 with class instead of structure with having the data members private and some member functions in private scope and some in public scope

3. Create a code snippet that illustrates the following: Calling of private member functions inside public member function

CODE:

```
#include <iostream>  
#include <string> using  
namespace std;  
  
class Student  
{  
private:  
    string name;  
    int roll_no;    float  
    cgpa;    string  
    degree;    string  
    hostel;
```

```

public: void addDetails()
{
    cout << "Enter name: ";    cin
    >> name;

    cout << "Enter roll number: ";
    cin >> roll_no;

    cout << "Enter CGPA: ";    cin
    >> cgpa;

    cout << "Enter degree: ";    cin
    >> degree;

    cout << "Enter hostel: ";
    cin >> hostel;
}

void updateDetails()
{
    cout << "Enter new name: ";
    cin >> name;

    cout << "Enter new roll number: ";
    cin >> roll_no;

```

```

Output
Enter name: Saanvi
Enter roll number: 4
Enter CGPA: 10
Enter degree: btech
Enter hostel: q

Student Details:
Name: Saanvi
Roll No: 4
CGPA: 10
Degree: btech
Hostel: q
Enter new name: roy
Enter new roll number: 4
Enter new CGPA: 8
Enter new degree: law
Enter new hostel: r

Student Details:
Name: roy
Roll No: 4
CGPA: 8
Degree: law
Hostel: r

=== Code Execution Successful ===

```

```
        cout << "Enter new CGPA: ";  
cin >> cgpa;
```

```
        cout << "Enter new degree: ";  
cin >> degree;
```

```
        cout << "Enter new hostel: ";  
cin >> hostel;  
    }
```

```
void displayDetails()  
{  
    cout << "\nStudent Details:\n";  
    cout << "Name: " << name << endl;  
    cout << "Roll No: " << roll_no << endl;  
    cout << "CGPA: " << cgpa << endl;    cout  
<< "Degree: " << degree << endl;    cout  
<< "Hostel: " << hostel << endl;  
}  
};
```

```
int main()  
{  
    Student s;
```

```
s.addDetails();  
s.displayDetails();  
  
s.updateDetails();  
s.displayDetails();  
  
return 0;  
  
}
```

4. Define a class Rectangle with variables width and height of integer typwe along with following:

(a) void getdata() to initialize object values

(b) void calculatearea() to calculate and display the area.

CODE

```
#include<iostream>  
using namespace std;  
class rectangle{ private:  
int width,height;  
public:  
void getdata(){  
cout<<"enter width and height of the rectangle: ";  
cin>>width>>height;
```

```

} void calculatearea(){
    int area=width*height;    cout<<"area of
rectangle is: "<<area;
    }
}; int main(){ rectangle rect;
    rect.getdata();
    rect.calculatearea();
    return 0;
}

```

Output

```

enter width and height of the rectangle: 4 5
area of rectangle is: 20

```

```

=== Code Execution Successful ===

```

5. Define a class Complex with variables real and imaginary along with following:

(a) void setComplex (float, float) to initialize object values.

(b) void displayComplex() to show the complex number

(c) Pass and return objects to calculate sum of two complex numbers. Display the sum

CODE:

```

#include <iostream>
using namespace std;

```

```

class Complex {
    float real, imag;

```

```

public:    void setComplex(float
r, float i)

```

```

    {
        real
    = r;
    imag = i;
    }

void displayComplex()
{
    cout << real << " + " << imag << "i" << endl;
}

```

```

Complex add(Complex c)
{
    Complex temp;
    temp.real
= real + c.real;
    temp.imag =
imag + c.imag;
    return temp;
}
};

```

```

int main()
{
    Complex c1, c2, sum;

    c1.setComplex(2.5, 3.5);
    c2.setComplex(1.5, 4.5);

    sum = c1.add(c2);

    sum.displayComplex();
}

```

```

Output
4 + 8i

=== Code Execution Successful ===

```

```
    return 0;
}
```

6. Implement scope resolution operator : : for the following uses:

- (a) Class functions defined outside the class**
- (b) Access a global variable with same name as a local variable**
- (c) Access a static variables**
- (d) Use inbuilt libraries (cin cout with scope resolution operator)**

-CODE:

```
#include <iostream> using
namespace std;
```

```
int value = 100;
```

```
class demo
{ private:
static int
count;
```

```
public:
    void show();
};
```

```
void demo::show()
{    int value = 50;    cout << "local value=
" << value << endl;    cout << "global value= " <<
```

Output

```
demonstration of scope resolution operator:
local value= 50
global value= 100
static count= 10
```

```
=== Code Execution Successful ===
```

```

::value << endl;    cout << "static count= " <<
demo::count << endl;
}

int demo::count = 10;

int main() {    demo d;    cout << "demonstration of scope
resolution operator:" << endl;    d.show();    return 0;
}

```

7. Create a code to implement the namespace and use similar variables and functions defined in different code sections.

CODE:

```

#include <iostream> using namespace std;

namespace first {    int value = 10;    void show()
{
    cout << "first namespace value: " << value <<
endl;
}
}

```

```

namespace second
{    int value =
20;    void
show()
{

```

Output

```

first namespace value: 10
second namespace value: 20
first value= 10
second value= 20

=== Code Execution Successful ===

```

```
        cout << "second namespace value: " << value << endl;
    }
}
```

```
int main() {    first::show();    second::show();    cout
<< "first value= " << first::value << endl;    cout <<
"second value= " << second::value << endl;    return
0;
}
```